



Módulo 4: Resolución actividad II

Parte 1 – Evaluación de diseño normalizado

1. Análisis crítico de la estructura dada Se ha proporcionado la siguiente estructura de base de datos:

CLIENTES(id_cliente, nombre, email, id_localidad)
VENTAS(id_venta, fecha, id_cliente, total)
DETALLES_VENTA(id_venta, id_producto, cantidad, precio_unitario)
PRODUCTOS(id_producto, descripcion, precio)
LOCALIDADES(id_localidad, nombre_provincia)

Análisis de claves primarias y foráneas:

- **CLIENTES:** id_cliente es la **clave primaria (PK)**. id_localidad es una **clave foránea (FK)** que referencia a LOCALIDADES.
- **VENTAS:** id_venta es la **PK**. id_cliente es una **FK** que referencia a CLIENTES.
- **DETALLES_VENTA:** (id_venta, id_producto) es la **clave primaria compuesta (PK)**. id_venta es una **FK** que referencia a VENTAS, y id_producto es una **FK** que referencia a PRODUCTOS.
- **PRODUCTOS:** id_producto es la **PK**.
- **LOCALIDADES:** id_localidad es la **PK**.

2. Diagnóstico de violaciones (si las hay) Se evalúa la estructura para detectar posibles violaciones a las tres primeras formas normales (1FN, 2FN y 3FN).

- **Primera Forma Normal (1FN):** Todas las tablas parecen cumplir con 1FN. No se observan atributos multivaluados ni atributos que contengan grupos repetitivos en ninguna de las tablas presentadas. Cada atributo contiene valores atómicos.
- **Segunda Forma Normal (2FN):** Una tabla está en 2FN si está en 1FN y todos sus atributos no clave son completamente dependientes de la clave primaria. Esto es relevante principalmente para tablas con claves primarias compuestas.
 - La única tabla con una clave primaria compuesta es **DETALLES_VENTA**(id_venta, id_producto, cantidad, precio_unitario).
 - cantidad depende de la combinación completa de (id_venta, id_producto).
 - **precio_unitario:** En el contexto de un sistema de ventas, precio_unitario generalmente representa el precio al que se vendió un producto en una transacción específica. Este precio puede variar con el tiempo o por ofertas, por lo que depende directamente de la combinación (id_venta, id_producto). Si fuera así, no habría violación de 2FN. Si, por el

contrario, precio_unitario se refiriera al precio actual del producto (redundante con PRODUCTOS.precio) y dependiera solo de id_producto, entonces sí violaría la 2FN. ***Asumiendo la interpretación más común de que registra el precio histórico de venta, no se detecta una violación de 2FN.***

- **Tercera Forma Normal (3FN):** Una tabla está en 3FN si está en 2FN y no existen dependencias transitivas de atributos no clave con la clave primaria (es decir, ningún atributo no clave depende de otro atributo no clave).
 - En todas las tablas (**CLIENTES, VENTAS, DETALLES_VENTA, PRODUCTOS, LOCALIDADES**), los atributos no clave parecen depender directamente de la clave primaria, sin dependencias transitivas.
 - Por ejemplo, en **LOCALIDADES(id localidad, nombre_provincia)**, nombre_provincia depende directamente de id_localidad. No hay otro atributo no clave del cual nombre_provincia dependa. Por lo tanto, **la tabla LOCALIDADES cumple con 3FN tal como está definida.**

Conclusión del Diagnóstico: La estructura de la base de datos entregada **aparenta estar correctamente normalizada hasta la Tercera Forma Normal (3FN)**, bajo las interpretaciones estándar de los atributos y el rol de precio_unitario como precio de venta histórico. No se identifican violaciones explícitas a 2FN o 3FN.

3. Propuesta de mejora o validación del modelo

- **Validación:** El modelo propuesto es **válido y cumple con los principios de normalización hasta 3FN.**
- **Propuesta de mejora (diseño conceptual):** Si bien la tabla **LOCALIDADES(id localidad, nombre_provincia)** es técnicamente en 3FN, desde un punto de vista de diseño conceptual, es atípico que la tabla LOCALIDADES no contenga un nombre_localidad (nombre del municipio/ciudad) y que nombre_provincia se relacione directamente con id_localidad. Una estructura más detallada y semánticamente clara, que mantendría la normalización, podría ser:
 - **PROVINCIAS(id_provincia, nombre_provincia)**
 - **LOCALIDADES(id_localidad, nombre_localidad, id_provincia_FK)**
 - **CLIENTES(id_cliente, nombre, email, id_localidad_FK)** Esta mejora optimizaría la representación de entidades geográficas, aunque añadiría otra tabla y un JOIN adicional para obtener la provincia del cliente. Sin embargo, dado el alcance de la actividad de validar la normalización existente, el modelo original es considerado correcto.

Parte 2 – Normalización inversa guiada (*Denormalización*)

1. Objetivo para la desnormalización Se elige el siguiente objetivo para la desnormalización:

- Reducir la cantidad de JOINS en reportes frecuentes y aumentar la velocidad de consultas en dashboards o informes.

2. Explicación de qué tablas se deciden fusionar o simplificar Se decide simplificar las consultas que requieren información de clientes y la provincia a la que pertenecen. Para lograr esto, se realizará una **desnormalización parcial incorporando el atributo nombre_provincia directamente en la tabla CLIENTES**.

- **Tabla seleccionada para modificación:** CLIENTES.
- **Información por incorporar:** *nombre_provincia* (originalmente de LOCALIDADES).

3. Justificación de la decisión desde el punto de vista funcional o de rendimiento

- **Contexto de uso:** En muchos sistemas, especialmente aquellos orientados a análisis de datos, reportes o dashboards, es muy común y frecuente que se necesite mostrar el nombre de la provincia junto con los datos del cliente (e.g., "Clientes por provincia", "Ventas por provincia de cliente").
- **Impacto de la normalización:** En el diseño normalizado actual, obtener el nombre_provincia para cada cliente requiere una operación JOIN entre CLIENTES y LOCALIDADES. Si estas consultas son muy frecuentes o involucran grandes volúmenes de datos, la sobrecarga de múltiples JOINS puede afectar negativamente el rendimiento del sistema.
- **Beneficio de la desnormalización:** Al incorporar nombre_provincia directamente en la tabla CLIENTES, se **elimina la necesidad de realizar este JOIN** para las consultas que solo necesitan el nombre de la provincia y la información del cliente. Esto **acelera significativamente el tiempo de respuesta** para dichas consultas, lo cual es altamente beneficioso en **sistemas de lectura intensiva** como los de reportes (OLAP, data warehouses). La desnormalización, en este caso, **prioriza la eficiencia en operaciones de lectura**.

Estructura desnormalizada propuesta:

- CLIENTES_DENORM(id_cliente, nombre, email, id_localidad, nombre_provincia)
- VENTAS(id_venta, fecha, id_cliente, total)
- DETALLES_VENTA(id_venta, id_producto, cantidad, precio_unitario)
- PRODUCTOS(id_producto, descripcion, precio)
- LOCALIDADES(id_localidad, nombre_provincia) (Esta tabla podría mantenerse para mantener la integridad de las provincias, pero su función principal de proporcionar nombre_provincia a CLIENTES sería redundante para las consultas que se desean optimizar).

4. Señalar las ventajas y desventajas de este cambio

- **Ventajas:**
 - **Mejora del Rendimiento en Consultas de Lectura:** Se reduce drásticamente el número de operaciones JOIN necesarias para obtener la información de la provincia del cliente en reportes y dashboards frecuentes, resultando en consultas más rápidas.
 - **Simplificación de Consultas:** Las sentencias SQL para obtener la información del cliente y su provincia son más directas y fáciles de escribir.
 - **Acceso Directo a Datos Combinados:** Ideal para escenarios donde se accede repetidamente a datos que son una combinación de CLIENTES y LOCALIDADES.
- **Desventajas:**
 - **Redundancia de Datos:** La principal desventaja es la introducción de redundancia; el nombre_provincia se almacena duplicado para cada cliente que pertenece a una localidad, y también existe en la tabla LOCALIDADES.
 - **Potencial de Inconsistencias:** Si el nombre_provincia en la tabla LOCALIDADES cambia (ej., por una corrección de un error tipográfico), este cambio debe propagarse a **todos** los registros de CLIENTES_DENORM que referencian esa localidad. Si no se realiza correctamente, los datos pueden volverse inconsistentes.
 - **Mayor Complejidad de Mantenimiento:** La gestión de datos redundantes puede ser más compleja, requiriendo mecanismos adicionales (como triggers o procedimientos almacenados) para asegurar que la información permanezca sincronizada y evitar anomalías de modificación.
 - **Incremento del Espacio de Almacenamiento:** Aunque menor en muchos casos, el almacenamiento de datos duplicados consume más espacio.

Discusión crítica.

La denormalización no debe verse como una mala práctica en sí misma, sino como una **decisión estratégica y justificada**. En este caso, la justificación es clara: la optimización del rendimiento para consultas y reportes de lectura intensiva. Para que esta decisión sea exitosa y sostenible, es fundamental seguir las **buenas prácticas de desnormalización**:

- **Justificación basada en métricas:** La decisión debería estar respaldada por métricas de rendimiento y consumo de recursos que demuestren la mejora.
- **Documentación:** Es crucial documentar los cambios realizados, las tablas fusionadas o simplificadas, las dependencias lógicas creadas y las razones para la desnormalización.
- **Mecanismos de Actualización:** Para mitigar el riesgo de inconsistencias, se deben implementar mecanismos de actualización automáticos (por ejemplo, mediante triggers o un proceso ETL si es un data warehouse) que garanticen que los datos redundantes se mantengan sincronizados cuando la información original (en este caso, nombre_provincia en LOCALIDADES) cambie.

- **Evaluación a largo plazo:** Se debe evaluar el impacto a largo plazo de este cambio en la mantenibilidad y la evolución del sistema.